

Something is wrong and we cannot save your story. [Learn more about how to fix this](#)

Models with Market Data for Equity Forecasting and Systematic Trading

This project aims to enhance short-term equity forecasting by integrating textual sentiment analysis with traditional market data. We use FinBERT, a language model fine-tuned on financial text, to extract structured sentiment signals from real-world headlines and align them with corresponding OHLCV (Open, High, Low, Close, Volume) data. By merging these two modalities, we construct a time-indexed dataset that supports supervised learning, to predict whether a stock should be bought, sold, or held on the next trading day. This hybrid framework enables the exploration of whether language-derived signals, when fused with price and volume dynamics, can systematically improve trading performance.

Web Scraping and Dataframe Processing

We begin by assembling two complementary data streams. First, through the Finnhub.io REST API we pull headline news for each target ticker—every timestamped headline, from press releases to analyst commentaries, is captured in real time. Second, we source historical OHLCV (Open, High, Low, Close, Volume) data via CSV downloads from Yahoo Finance, and supplement this with a lightweight scraper for intraday quotes when live pricing is needed. This dual ingestion ensures that our universe of raw inputs spans both the narrative sentiment that moves markets and the hard price-volume facts against which it plays out.

We forward fill days with missing data—whether from the financial side (e.g., market closed, no OHLCV recorded) or from the sentiment side (e.g., no headlines published).

FinBERT Sentiment Analysis

With clean data frames ready, we invoke FinBERT—a BERT variant fine-tuned on millions of financial sentences—to transform raw headlines into quantitative sentiment features. In our `SentimentAnalyzer` class, the constructor instantiates a high-

throughput NLP pipeline. Each headline batch is passed through this pipeline, yielding three posterior probabilities:

$$(P_+, P_-, P_0)$$

for positive, negative, and neutral sentiment. We normalize these scores to ensure they sum to unity, and compute a continuous sentiment score

$$s = P_+ - P_-.$$

A simple threshold on $s(\pm 0.05)$ assigns each headline a discrete category (“bullish,” “bearish,” or “neutral”) while the maximum posterior defines a model-confidence metric. Because headlines arrive at arbitrary times, we “floor” each timestamp to its trading date t and aggregate per-day statistics: the arithmetic mean of s , the mean of each class probability, and the headline count n_t . This condenses a high-frequency, irregular stream into a daily sentiment vector

$$\mathbf{s}_t \triangleq [\bar{s}_t, \bar{P}_t^+, \bar{P}_t^-, \bar{P}_t^0, n_t]$$

Concurrently, we ingest the corresponding OHLCV time series from a standard financial data source. We inner-join this market data to the sentiment aggregates on the calendar date, retaining only those days present in both sources. The merged dataset thus has dimension $T \times d$, where $d = 5$ financial features + 5 sentiment features = 10 total predictors.

To derive our target variable, we compute the one-step ahead return

$$\mathbf{r}_t = (C_{t+1} - c_t)/C_t,$$

where C_t is the closing price on day t . We then apply symmetric thresholds (here $\pm 0.1\%$) to discretize returns into a three-class label function:

$$y_t = \begin{cases} \text{"buy"}, & r_t > +0.001, \\ \text{"sell"}, & r_t < -0.001, \\ \text{"hold"}, & |r_t| \leq 0.001. \end{cases}$$

By shifting the return series backwards by one index and dropping the terminal NaN, we align each feature vector x_t with its corresponding label y_t , yielding a clean supervised learning dataset.

Feature Engineering

Once the joint dataset of price action and FinBERT-generated sentiment scores is assembled and chronologically ordered, we proceed to a feature engineering phase aimed at enriching the raw inputs with derived quantities that encode market dynamics, momentum structures, and behavioral interactions. Several classes of features are constructed. First, we introduce lagged differences in sentiment to capture momentum in news tone—specifically, we compute a 3-day difference in both the raw sentiment score and the net count of bullish versus bearish articles, yielding

$$\Delta s_t^{(3)} = s_t - s_{t-3}, \text{ and } \Delta n_t^{\text{net}} = n_t^{\text{bull}} - n_t^{\text{bear}} - (n_{t-3}^{\text{bull}} - n_{t-3}^{\text{bear}}).$$

These differences aim to proxy whether sentiment is accelerating positively or deteriorating. We then compute a 5-day simple moving average of the sentiment score,

$$\bar{s}_t^{(5)} = \frac{1}{5} \sum_{i=0}^4 s_{t-i},$$

to smooth out transient fluctuations and better capture prevailing sentiment regimes. Additionally, to incorporate liquidity-aware sentiment weighting, we normalize each day's volume by its trailing 20-day average and define a volume-weighted sentiment factor

$$s_t^{VW} = s_t \cdot \left(\frac{V_t}{\bar{V}_t^{(20)}} \right).$$

This allows us to upweight sentiment signals on days where the stock is unusually active, which may indicate broader market participation or reaction.

From a purely technical standpoint, we introduce a binary “moving average crossover” feature—equal to 1 on days when the short-term price average (5-day) crosses above the medium-term trend (20-day), a widely used heuristic for regime shift detection. We also define a boolean feature indicating whether the direction of the sentiment score and the daily return agree in sign. The goal of this variable is to be predictive of self-fulfilling behavior, where sentiment-congruent price action reflects reinforcing crowd dynamics.

Finally, we introduce a simple categorical feature representing the day of the week ($t \bmod 7$), to capture any calendar-based seasonality or anomalies that might influence intraday returns, such as Monday underperformance or Friday rallies. Due to the use of rolling windows and lag operations, the first few rows of the dataset are necessarily dropped as they contain NaN values from incomplete lookback windows.

Train/Test Split

Following the feature engineering stage, we isolate a subset of informative predictors and prepare the dataset for supervised classification. The feature columns define the model’s input vector x_t , comprising of 21 dimensions drawn from three distinct categories. The first group consists of traditional price-based technical indicators: the daily return r_t (defined above), three rolling averages (MA5, MA10, MA20), the Relative Strength Index (RSI), and raw trading volume. The second group captures FinBERT-generated sentiment attributes, including the continuous sentiment score s_t , class probabilities (bullish, bearish), discrete article counts by polarity, net sentiment (bullish minus bearish), and the bull–bear ratio—a measure of directional imbalance. The third group encompasses engineered features discussed previously, including

momentum, volume-normalized sentiment, crossover signals, and calendar effects.

The binary classification target is whether the price will be up the next day. To preserve the temporal ordering of financial time series and prevent data leakage, the dataset is split deterministically using a time-aware cutoff. Specifically, the first 80% of observations are used for training, while the remaining 20% constitute the test set, enabling evaluation on unseen future data.

Prior to modeling, we standardize each feature to have zero mean and unit variance using scikit-learn's StandardScaler. This ensures numerical stability during optimization and guards against dominance of high-magnitude predictors like trading volume. The result is two aligned matrices of sizes [66 x 21] (test) and [17 x 21] (train) respectively, each paired with target vector y .

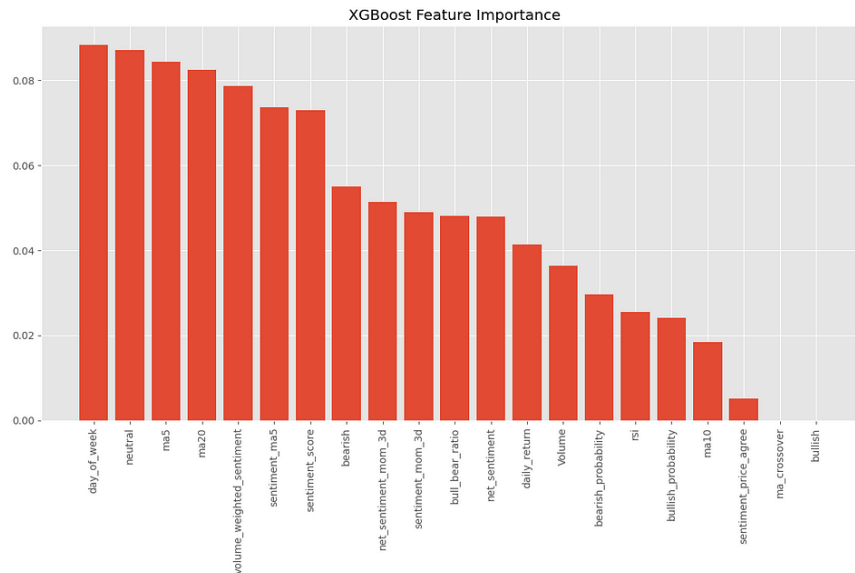
Training and Evaluation

In this stage, we benchmark two ensemble classifiers: RandomForest and XGBoost. After fitting, we evaluate each model's predictive performance on the held-out test set via classification accuracy, ROC-AUC (Area Under the Receiver Operating Characteristic curve), and the full precision/recall/F1 classification report.

Looking at Boeing empirically, XGBoost outperforms Random Forest with an accuracy of 70.6% and an ROC-AUC of 0.742, compared to Random Forest's 58.8% accuracy and higher—but potentially overfitted—ROC-AUC of 0.826. These results suggest that XGBoost is more effective in capturing the underlying signal in this moderately imbalanced setting, especially given its superior macro-averaged F1 score and more balanced precision–recall tradeoff. Notably, both models skew toward correctly predicting the positive class (price increases), which aligns with our labeling strategy that defines “price_up_next_day” as the target.

In addition to raw predictive performance, we inspect each model's internal ranking of input features by plotting the learned importance scores. Random Forest's Gini-based importances and XGBoost's gain-based scores both highlight the dominance of longer-term trend indicators (like MA20), recent sentiment momentum, and bullish

probability, indicating that both price memory and directional textual sentiment contribute materially to the decision boundary. These insights reinforce the hybrid nature of the signal and guide future feature selection or refinement.



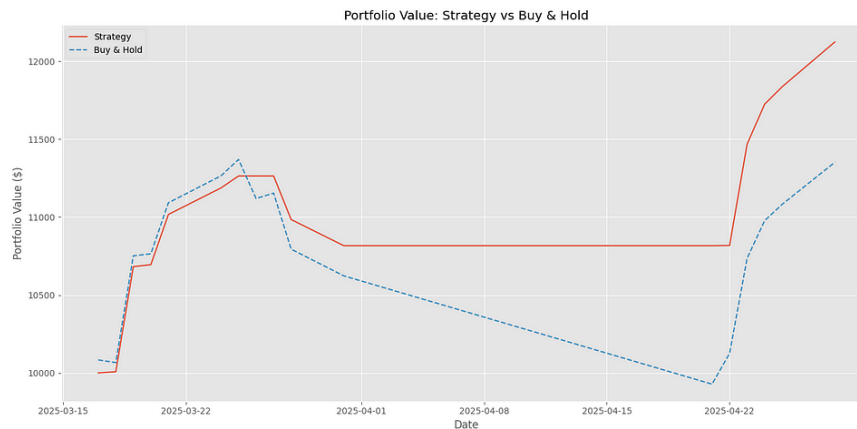
Backtesting

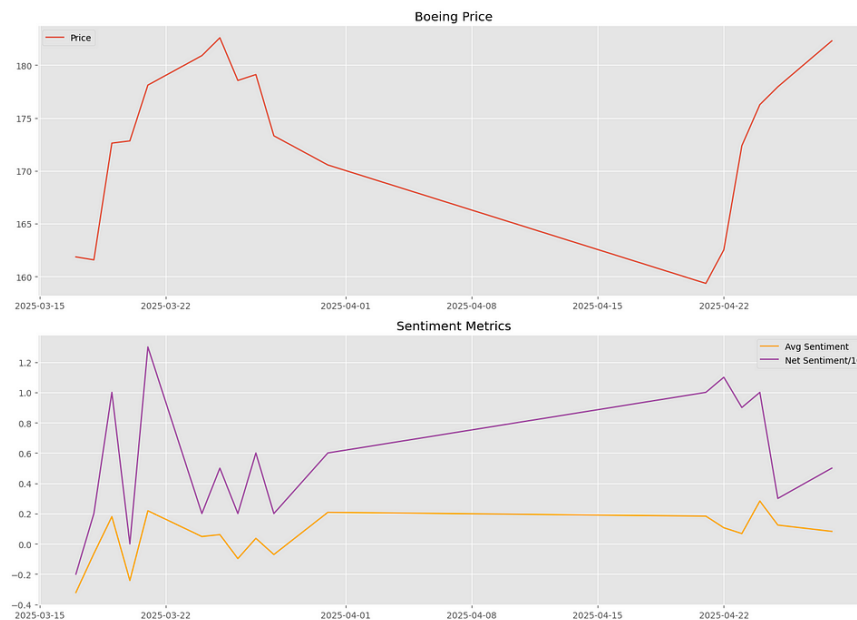
To evaluate the real-world viability of the trained classifier, the notebook proceeds by generating discrete trading signals from the predicted class probabilities of the XGBoost model. Rather than using the raw binary prediction, the decision threshold is set to a configurable value (defaulting to 0.55) to allow for confidence-weighted signaling. Specifically, if the predicted probability of a price increase on the next day exceeds 55%, the model emits a “BUY” signal; if it falls below 45% (i.e., $1 - 0.55$), the signal becomes “SELL”; values in the intermediate band are assigned a “HOLD”.

The downstream economic utility of these predictions is then assessed via a simplified yet robust backtesting engine. The strategy is initialized with a fixed notional cash balance (e.g., \$10,000) and executes trades one day after the signal is emitted using the subsequent day’s opening price, thereby avoiding look-ahead bias. A BUY signal triggers a full allocation of available capital into the asset, purchasing integer shares; a SELL signal liquidates all held shares at market open; a HOLD signal maintains the current portfolio

composition. Daily portfolio equity is recalculated as the sum of cash and the market value of held shares, with returns and cumulative equity tracked throughout the test window.

To contextualize performance, a passive Buy-and-Hold benchmark is constructed using the same initial capital, applied on the first day of the test period. Performance metrics include total return, maximum drawdown (peak-to-trough loss), and the annualized Sharpe ratio, computed as the mean daily return over its standard deviation scaled by $\sqrt{252}$, the number of trading days in a year. In this experiment, the active sentiment-driven strategy achieves a total return of +21.23%, outperforming the buy-and-hold benchmark at +13.52%, with a notably high Sharpe ratio of 8.06, indicating exceptional risk-adjusted performance. Furthermore, the strategy's maximum drawdown is limited to just -3.97%, suggesting that its tactical responsiveness helped preserve capital during short-term dips.





Monte Carlo Simulation and Lagged Sentiment Rates

Next, we turned our attention toward assessing the generalizability and temporal robustness through stochastic Monte Carlo simulation. The core idea is to evaluate how shifts in the timing between sentiment expression and market response (i.e., sentiment “lags”) affect downstream trading profitability and risk-adjusted performance. This approach moves beyond deterministic backtesting by injecting variability into the feature set—specifically, by randomly varying the lag applied to sentiment-based predictors.

To implement this, we define a lag window $[1, L]$ (in this case $L = 5$) and randomly sample a lag value and randomly sample a lag value ℓ from this range in each simulation. For each trial, we shift all sentiment-derived features (e.g., raw score, net sentiment, bullish count, etc.) by ℓ days backward, creating lagged features like $\text{sentiment_score}_{\{t-\ell\}}$, which now attempt to explain returns ΔP_t at a delay. This models the hypothesis that market reactions to textual sentiment may not be instantaneous but could manifest after a short lead time.

After lagging and aligning the data (discarding early rows where lag-induced NaNs occur), we isolate training and testing sets using a time-aware split. The same XGBoost classifier is then retrained per simulation on this new lagged feature configuration. Probabilistic

predictions are translated into buy/sell/hold signals via a thresholding function (e.g., buy if $P(y = 1) > 0.55$), and passed through our rolling backtest engine.

The backtest tracks portfolio evolution under a basic position-sizing strategy: when a “BUY” is signaled, all available cash is used to purchase shares at the next open price; when a “SELL” is triggered, all holdings are liquidated. This simplistic rule allows us to directly map model outputs to trade execution. After simulation s , we record total return

$$\left(\frac{\text{final equity}}{\text{initial capital}} - 1 \right),$$

benchmark buy-and-hold return, annualized Sharpe ratio

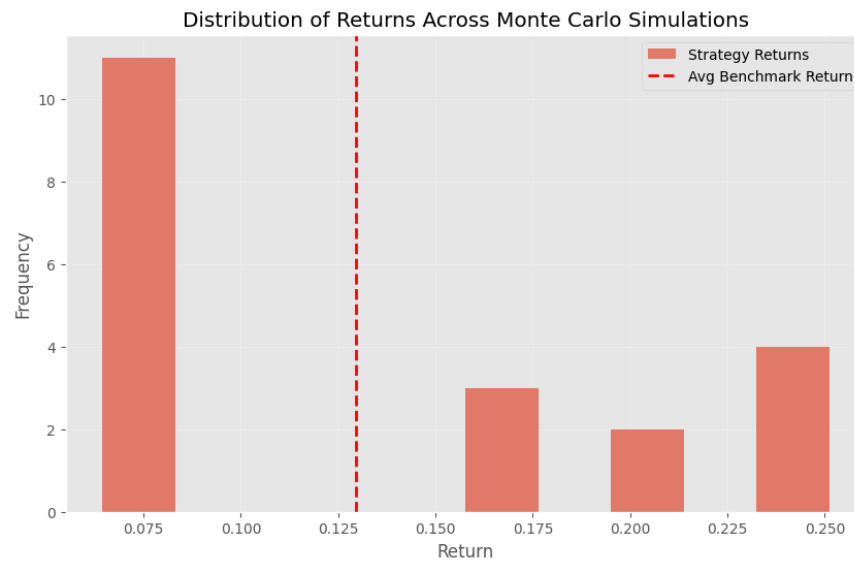
$$\left(\frac{\mu_r}{\sigma_r} \cdot \sqrt{252} \right),$$

and maximum drawdown

$$\min \left(\frac{P_t}{\max_{t' \leq t} P_{t'}} - 1 \right),$$

yielding a comprehensive tuple of performance metrics.

Over 20 simulation runs, each with independently sampled lag values, we observe substantial variability in strategy returns—from a low of ~6.4% to a high of ~25.1%—with an average return of ~13.6%. Notably, 45% of simulations outperformed the static benchmark return of ~13.5%, indicating that sentiment lags, when correctly aligned, can confer predictive edge. The best-performing strategies tend to emerge at lags of 1–2 days, as seen in the boxplots of return and Sharpe ratio by lag. This supports a hypothesis of short-term market memory: textual sentiment may not move prices immediately but can induce measurable drift in subsequent sessions.



Furthermore, Sharpe ratios in the top simulations reach values exceeding 9.7, while drawdowns remain modest ($\sim 3\%$), reflecting high return consistency relative to volatility. In contrast, longer lags (lags ≥ 4) often degrade performance, suggesting diminishing influence of stale sentiment signals.





In the performance-by-lag analysis, we distilled the results of our Monte Carlo simulations into a succinct DataFrame that reports, for each sentiment lag from one to five days, the average strategy return, Sharpe ratio, maximum drawdown, and the proportion of simulations that outperformed the benchmark. By iterating over the Monte Carlo results and grouping by lag, we compute

$$\overline{R}_l = \frac{1}{N_l} \sum_{i=1}^{N_l} R_i, \overline{S}_l = \frac{1}{N_l} \sum_{i=1}^{N_l} S R_i, \overline{D}_l = \frac{1}{N_l} \sum_{i=1}^{N_l} D_i, P_l = \frac{1}{N_l} \sum_{i=1}^{N_l} (R_i > B_i),$$

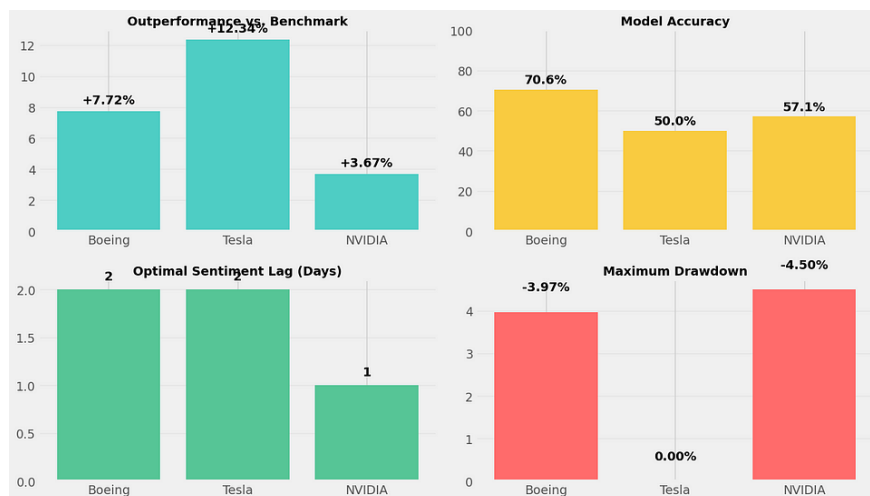
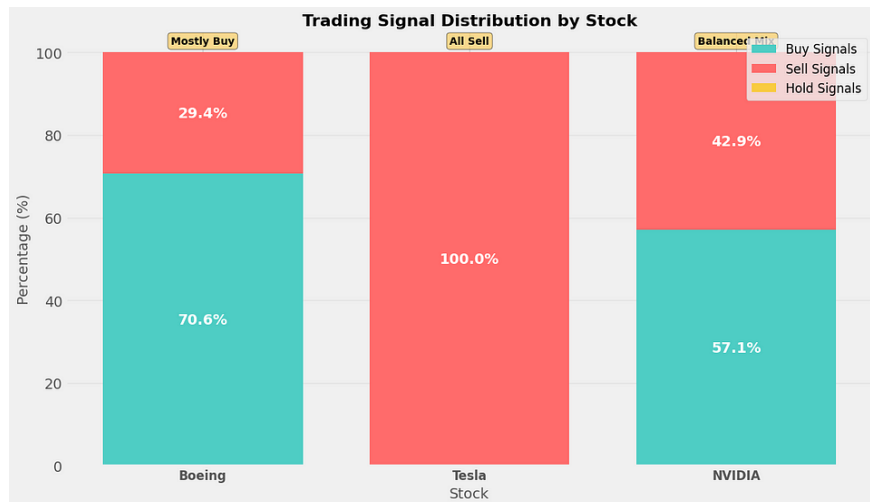
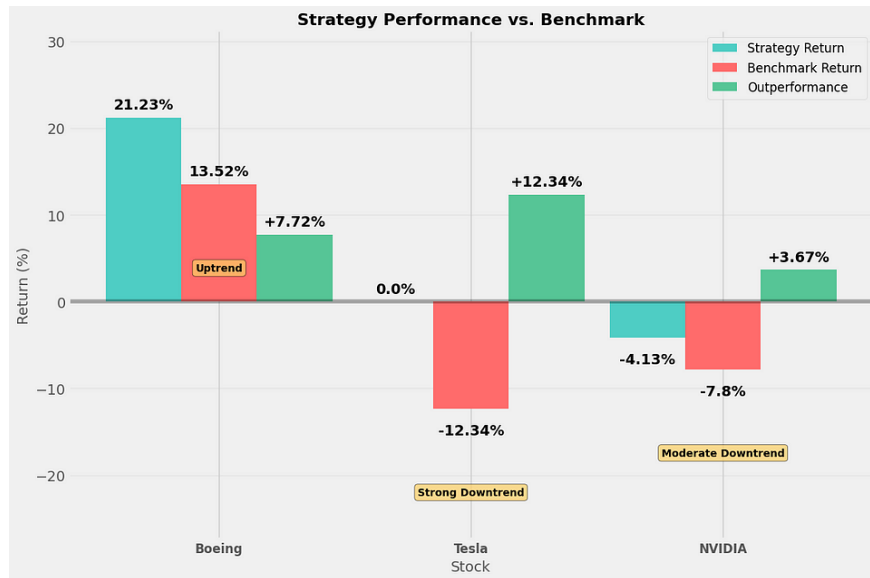
where R_i , SR_i , and D_i denote the return, annualized Sharpe, and maximum drawdown for simulation i with lag ℓ (ℓ in the equation), and B_i the corresponding benchmark return. Plotting R_ℓ across lags reveals a clear peak at a two-day delay—suggesting that two-day-lagged sentiment yields the highest average return ($\approx 25.1\%$) and the smallest average drawdown ($\approx -3.0\%$), while one- and five-day lags also consistently beat the benchmark but to a lesser extent. In contrast, three- and four-day lags underperform, exhibiting both lower returns and deeper drawdowns.

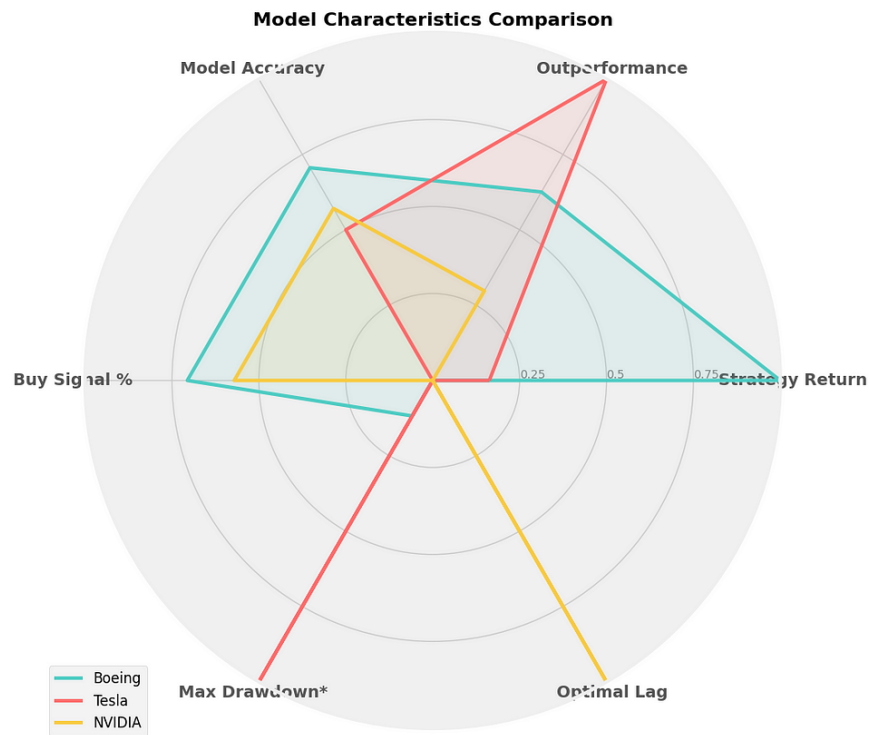
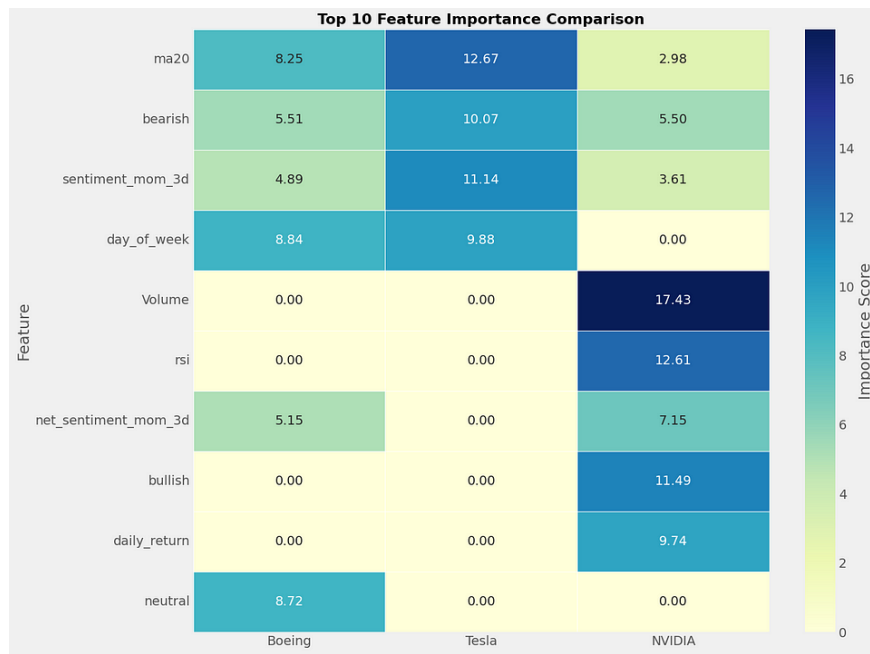
Complementing this, we examined how the distribution of trading signals evolves with lag. For each lag ℓ , we averaged across simulations the fraction of buy, sell, and hold decisions,

$$\pi_l^{\text{buy}} = \frac{1}{N_l} \sum_i \frac{\# \text{buy signals}}{T}, \pi_l^{\text{sell}}, \pi_l^{\text{hold}},$$

where T is the number of test-period days. The bar chart reveals that as lag increases, the model issues progressively fewer buy signals and more sell or hold recommendations, reflecting a more cautious stance when sentiment is older. At a two-day lag—our identified optimum—the strategy still issues 88 % buys and 12 % sells, striking a balance between responsiveness and noise reduction.

Twenty simulations spanned lags of one to five days, yielding an average strategy return of 13.6 % versus a 13.0 % benchmark return. The best-performing simulation achieved +25.1% (lag = 2), while the weakest returned +6.4% (lag = 4). Overall, 45 % of simulations outperformed buy-and-hold, and a two-day lag emerged as the most robust predictor. We conclude that sentiment shifted two trading days prior imparts the greatest anticipatory power, motivating a recommendation to construct signals exclusively from two-day-lagged sentiment while enforcing stop-loss rules to cap drawdowns.





* For Max Drawdown, higher value means smaller drawdown

Auto (C#) ▾

Signal distribution by lag:

	Lag	Buy %	Sell %	Hold %
0	1	94.117647	5.882353	0.00
1	2	88.235294	11.764706	0.00
2	3	81.250000	18.750000	0.00

3	4	75.000000	18.750000	6.25
4	5	68.750000	12.500000	18.75